

PLATEFORME INTELLIGENTE DE COLLECTE ET DE MONITORING
DES DONNÉES ISSUES DES TRAITEMENTS DES EAUX USÉES

Rapport Technique

Module Monitoring — Conception, Architecture
et Analyse Scientifique

Réalisé par

Hajar Tarfouss Aya Bourhdifa

Module

Tâche 2 — Monitoring
Dashboard
(de A à Z)

PÉRIMÈTRE DE CE DOCUMENT

Ce rapport couvre exclusivement le module de *monitoring* : le service `monitoring_backend` (FastAPI) et le tableau de bord `monitoring_frontend` (Next.js). La collecte des données brutes (ThingSpeak), le stockage en base PostgreSQL et la gestion des études/cycles sont assurés par le backend de l'équipe stockage, développé séparément et consommé ici exclusivement via son API REST.

10 septembre 2026

Table des matières

1	Introduction et contexte	3
1.1	Objectifs du module	3
1.2	Contrainte architecturale fondamentale	3
1.3	Sources de données prévues	4
2	Architecture générale	5
2.1	Vue d'ensemble	5
2.2	Flux de données	5
3	Choix technologiques	7
3.1	Backend — <code>monitoring_backend</code>	7
3.1.1	Choix de conception : l'autoencodeur sans PyTorch	7
3.2	Frontend — <code>monitoring_frontend</code>	8
4	Architecture logicielle détaillée	9
4.1	Organisation du backend	9
4.1.1	<code>storage_client.py</code> — la couche d'accès unique	9
4.1.2	<code>json_safety.py</code> — robustesse des réponses API	9
4.1.3	Modules de services (<code>app/services/</code>)	10
4.2	Organisation du frontend	10
5	Le tableau de bord : fonctionnalités et méthodes	11
5.1	Monitoring en temps réel	11
5.2	Statistiques globales	11
5.3	Analyse scientifique (données temps réel)	12
5.3.1	Histogrammes	12
5.3.2	Corrélations (heatmap)	12
5.3.3	Boxplots	12
5.3.4	Timeline (tendance)	12
5.4	Comparaison	12
5.4.1	SET 1 vs SET 2	13
5.4.2	IN vs OUT_CONTROL vs OUT_PLANT	13
5.5	Alertes	13
5.5.1	Détection intelligente d'anomalies	13
5.5.2	Captures nulles consécutives	13
5.6	Journal	14
5.7	Qualité de l'eau	14
5.8	Études et cycles	14
5.8.1	Rappel du protocole expérimental	14
5.8.2	Onglet « Par cycle » — carte par paramètre	14
5.8.3	Onglet « Progression multi-cycles »	15
5.8.4	Onglet « Statistiques globales »	15
5.8.5	Onglets « Histogramme » et « Heatmap de corrélation »	15

5.9	Tableau de données	15
6	Récapitulatif des méthodes statistiques	16
6.0.1	Analyse en Composantes Principales (ACP)	16
7	Limites connues et perspectives d'amélioration	18
8	Conclusion	19

CHAPITRE 1

Introduction et contexte

Le projet global vise à mettre en place une plateforme intelligente de collecte et de suivi de la qualité des eaux usées traitées par phyto-épuration. Le travail est réparti entre plusieurs équipes ; ce rapport documente exclusivement le module dont j'ai eu la charge de bout en bout : le **monitoring** — c'est-à-dire tout ce qui transforme les données brutes collectées par l'équipe stockage en informations exploitables (statistiques, visualisations, alertes, comparaisons scientifiques).

1.1 Objectifs du module

- Visualiser en temps réel les mesures des capteurs de qualité d'eau (pH, température, conductivité électrique, turbidité, oxygène dissous) ;
- Calculer des statistiques descriptives et avancées sur ces mesures ;
- Détecter automatiquement les anomalies, par plusieurs méthodes complémentaires (statistique et machine learning) ;
- Comparer les performances de traitement entre plusieurs configurations expérimentales (eau brute, série témoin, série plantée) ;
- Fournir un journal chronologique des incidents (captures manquantes, anomalies détectées) ;
- Exposer l'ensemble de ces informations dans un tableau de bord web clair et exploitable par un non-développeur.

1.2 Contrainte architecturale fondamentale

Une règle a été fixée dès le départ par le chef de projet et respectée strictement tout au long du développement :

POINT D'ATTENTION

Le service de monitoring ne se connecte jamais directement à PostgreSQL. Toute donnée transite exclusivement par l'API REST du backend de l'équipe stockage.

Cette contrainte évite qu'une équipe modifie le schéma de la base sans prévenir les autres, empêche les accès concurrents mal maîtrisés, et permet de développer, tester et redémarrer le monitoring de façon totalement indépendante du reste du système.

1.3 Sources de données prévues

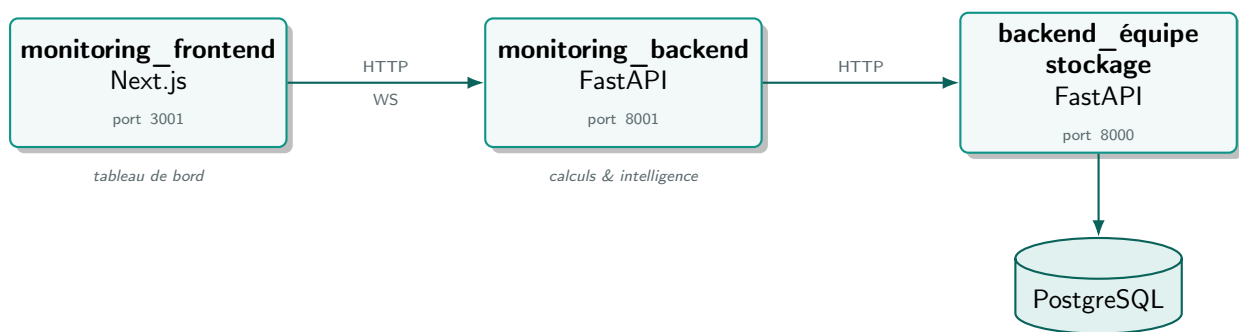
- Capteurs de qualité (temps réel) : pH, température, conductivité électrique (EC), oxygène dissous (DO), turbidité — collectés toutes les 5 minutes via l'API ThingSpeak, sur plusieurs sets de capteurs.
- Capteur multiparamètre (mesures plus fiables) : COD, BOD, TSS, température, pH, EC, DO, turbidité — mesurées manuellement, chaque mesure répliquée 3 fois. Suit un protocole en cycle : une première mesure sur l'eau usée brute (IN), traitée en parallèle dans une série de contrôles (3, sans plante) et une série de plantes (3, avec plante), avec une mesure sur chaque sortie (OUT-CONTROL et OUT-PLANT).

CHAPITRE 2

Architecture générale

2.1 Vue d'ensemble

Le système repose sur trois briques indépendantes communiquant uniquement par HTTP (et WebSocket pour le flux temps réel) :



backend_équipe_stockage

Seule application autorisée à écrire dans PostgreSQL. Responsable de la collecte périodique des données ThingSpeak, de la gestion des études et cycles de laboratoire, et de la détection des alertes de captures nulles consécutives.

monitoring_backend

Service de **calcul pur** : interroge l'API stockage, effectue tous les traitements scientifiques (statistiques, corrélations, détection d'anomalies, comparaisons), et ne persiste rien — tout est recalculé à la demande.

monitoring_frontend

Tableau de bord web. N'exécute aucun calcul scientifique ; se contente d'afficher les résultats déjà calculés par le **monitoring_backend**.

2.2 Flux de données

- 1 Les capteurs envoient leurs mesures à ThingSpeak.
- 2 Le backend stockage collecte ces mesures toutes les 5 minutes (planificateur APScheduler) et les insère en base PostgreSQL.

- 3 Le `monitoring_backend` interroge périodiquement (toutes les 15 secondes) l'API stockage pour détecter une nouvelle mesure, et la diffuse en temps réel aux clients connectés via WebSocket (`/ws/live`).
- 4 Pour les calculs plus lourds (statistiques, ANOVA, détection d'anomalies), le frontend interroge directement les endpoints REST dédiés du `monitoring_backend`, qui va chercher les données brutes nécessaires auprès de l'API stockage, effectue le calcul, et renvoie un résultat déjà prêt à afficher.

CHAPITRE 3

Choix technologiques

3.1 Backend — `monitoring_backend`

Outil	Rôle	Justification
FastAPI	Framework serveur	Même framework que l'équipe stockage (cohérence de style). Typé, rapide, asynchrone — indispensable pour le flux temps réel WebSocket sans bloquer le serveur.
httpx	Client HTTP	Supporte l'asynchrone, contrairement à <code>requests</code> , utile pour le <i>poller</i> de fond qui ne doit jamais bloquer les requêtes entrantes.
pandas	Manipulation de données	Standard pour les séries temporelles multi-paramètres (tri chronologique, moyenne mobile, agrégations par groupe).
NumPy	Calcul numérique	Histogrammes, tableaux pour l'entraînement des modèles de machine learning.
SciPy	Tests statistiques	Corrélations de Pearson/Spearman, ANOVA, test t de Welch, test de Kolmogorov-Smirnov, test de Shapiro-Wilk, loi de Student.
scikit-learn	Machine Learning	Isolation Forest, standardisation (<code>StandardScaler</code>), ACP (PCA), <code>MLPRegressor</code> utilisé comme autoencodeur.
statsmodels	Test post-hoc	Uniquement pour le test de Tukey HSD après l'ANOVA (non disponible dans SciPy).
websockets	Temps réel	Diffusion des nouvelles mesures au frontend sans recourir à un <i>polling</i> HTTP agressif côté client.

3.1.1 Choix de conception : l'autoencodeur sans PyTorch

Le cahier des charges demande explicitement un autoencodeur pour la détection d'anomalies. Un autoencodeur compresse une observation dans un espace de dimension réduite puis tente de la reconstruire ; une observation anormale se reconstruit mal, produisant une erreur de reconstruction élevée. Ce comportement est obtenu ici avec un `MLPRegressor` de scikit-learn configuré en goulot d'étranglement (architecture symétrique, par exemple $10 \rightarrow 5 \rightarrow 2 \rightarrow 5 \rightarrow 10$ neurones), entraîné à reproduire son entrée en sortie. Ce choix évite une dépendance lourde (PyTorch représente plusieurs centaines de mégaoctets et complique l'installation) pour un problème qui

reste, en pratique, à quelques dizaines de dimensions et quelques milliers de points — sans que le principe algorithmique ne soit modifié.

3.2 Frontend — `monitoring_frontend`

Outil	Rôle	Justification
Next.js 14 + TypeScript	Framework & typage	Le typage statique évite des erreurs d'intégration (ex. envoyer une chaîne de caractères là où l'API attend un nombre) directement à la compilation.
TailwindCSS	Style	Interface cohérente sans multiplier les fichiers CSS ; palette de couleurs personnalisée (tons <i>teal</i> /ardoise) pour une identité visuelle de type « salle de contrôle », plutôt qu'un thème générique.
Plotly.js (<code>plotly.js-dist-mi</code>)	Graphiques	Une seule bibliothèque couvre tous les types de graphes nécessaires (courbes, histogrammes, heatmaps, boxplots), avec export PNG natif. Le <i>bundle</i> pré-compilé pour navigateur est utilisé plutôt que le paquet complet, qui référence des modules Node.js incompatibles avec le empaqueteur de Next.js.
SWR	Gestion des appels API	Cache, re-récupération automatique, gestion des états de chargement/erreur, sans dupliquer la logique <code>useEffect/useState</code> sur chaque page.
lucide-react	Icônes	Bibliothèque légère et cohérente visuellement.

CHAPITRE 4

Architecture logicielle détaillée

4.1 Organisation du backend

```
monitoring_backend/  
  app/  
    config.py           Configuration (.env), réglages ML  
    norms_config.py     Seuils de conformité MA/EU  
    json_safety.py      Filet de sécurité JSON (NaN/Infinity -> null)  
    storage_client.py    SEULE couche d'accès aux données  
    poller.py           Boucle de fond (WebSocket temps réel)  
    main.py             Point d'entrée FastAPI  
    routes/             Endpoints (orchestration fine)  
    services/           Logique métier / scientifique
```

4.1.1 storage_client.py — la couche d'accès unique

Ce fichier centralise **tous** les appels vers l'API de l'équipe stockage : aucun autre fichier du projet n'a le droit d'appeler cette API directement. Un point technique important a nécessité une attention particulière : l'API stockage ne permet pas de filtrer les données brutes (**raw-data**) par date — seulement par **set_number**, avec une pagination **skip/limit** plafonnée à 1000 lignes par appel, triée du plus récent au plus ancien.

Pour permettre malgré tout un filtrage par période dans le tableau de bord, la fonction `fetch_raw_data_range()` pagine automatiquement : elle récupère des blocs de 1000 lignes, s'arrête dès que le bloc contient des dates antérieures à la date de début demandée, puis filtre précisément le résultat côté monitoring. Un garde-fou (`STORAGE_MAX_PAGES`) empêche toute boucle infinie sur une période trop large. La même logique de pagination protège également la récupération des alertes (`list_alerts`), l'API stockage plafonnant aussi sa limite à 1000 éléments par requête.

4.1.2 json_safety.py — robustesse des réponses API

Certains calculs statistiques peuvent produire des valeurs **NaN** ou **Infinity** (par exemple l'écart-type d'un groupe ne contenant qu'une seule valeur, ou un pourcentage d'élimination avec un dénominateur proche de zéro). Le moteur de sérialisation JSON utilisé par défaut par FastAPI refuse ces valeurs, provoquant une erreur serveur même quand le calcul sous-jacent est correct. Un filtre global (`SafeJSONResponse`) a été mis en place pour remplacer récursivement toute valeur **NaN/Infinity** par **null** avant l'envoi de **toute** réponse de l'API, protégeant ainsi les endpoints existants comme futurs sans avoir à traiter ce cas dans chaque service individuellement.

4.1.3 Modules de services (app/services/)

calculations.py

Statistiques descriptives de base, corrélations de Pearson/Spearman, détection d'outliers (règle IQR et Z-score), moyenne mobile, rapport de qualité des données, histogramme, résumé boxplot — pour les données temps réel des capteurs.

anomaly_service.py

Les trois méthodes de détection d'anomalies (Z-score, Isolation Forest, Autoencoder), le test de dérive entre les deux sets de capteurs, et la fonction `detect_anomaly_events()` qui unifie les trois méthodes avec une cause lisible pour le calendrier du journal.

comparison_service.py

Tout ce qui concerne les cycles de laboratoire : transformation des données en tableau exploitable, statistiques par étape du protocole, ANOVA et test de Tukey, analyse en composantes principales, statistiques globales étendues, boxplots, histogrammes, heatmap de corrélation, calcul du pourcentage d'élimination, suivi de la progression multi-cycles.

journal_service.py

Fusionne les alertes de captures nulles et les événements d'anomalies détectés par jour civil, pour l'affichage du calendrier.

4.2 Organisation du frontend

```
monitoring_frontend/
  app/
    page.tsx                "/"                Monitoring temps reel
    statistiques/           "/statistiques" Statistiques globales
    analyse/               "/analyse"         Analyse scientifique
    comparaison/           "/comparaison" Comparaison sets & IN/OUT
    alertes/               "/alertes"         Alertes
    journal/               "/journal"         Calendrier
    qualite/               "/qualite"         Qualite de l'eau
    etudes/               "/etudes"         Etudes & Cycles
    donnees/               "/donnees"        Tableau de donnees
  lib/api.ts               Centralise tous les appels HTTP
  components/PlotlyChart.tsx Chargement dynamique cote navigateur
  hooks/useLiveSocket.ts   Connexion WebSocket + reconnexion auto
```

Un point technique notable : Plotly.js manipule directement l'objet `window` du navigateur, ce qui est incompatible avec le rendu côté serveur de Next.js. Le composant `PlotlyChart.tsx` charge donc la bibliothèque de façon dynamique, exclusivement côté client.

CHAPITRE 5

Le tableau de bord : fonctionnalités et méthodes

Cette section décrit chaque page du tableau de bord, ce qui y est affiché, et — point central de ce rapport — les méthodes de calcul exactes utilisées derrière chaque indicateur.

5.1 Monitoring en temps réel

Affiche, pour chaque set de capteurs, la dernière mesure reçue pour les cinq paramètres suivis (pH, température, conductivité électrique, turbidité, oxygène dissous), avec un badge de conformité, ainsi que des courbes historiques filtrables par set, plante et période, exportables en image PNG et en CSV.

Conformité aux normes. — Pour chaque valeur, la conformité est évaluée par comparaison à des seuils de référence :

$$\begin{aligned}\text{conforme}_{\text{Maroc}} &= (\text{valeur} \geq \text{seuil}_{\min}^{MA}) \wedge (\text{valeur} \leq \text{seuil}_{\max}^{MA}) \\ \text{conforme}_{\text{Europe}} &= (\text{valeur} \geq \text{seuil}_{\min}^{EU}) \wedge (\text{valeur} \leq \text{seuil}_{\max}^{EU})\end{aligned}$$

Le badge affiché résulte de la combinaison de ces deux booléens : conforme aux deux normes, à une seule, ou à aucune.

POINT D'ATTENTION

Aucune valeur seuil officielle n'a été communiquée pour ce projet à ce stade. Les seuils actuellement utilisés sont des valeurs indicatives construites à partir de textes réglementaires publics traitant de sujets proches (NM 03.7.001, arrêté conjoint n°1276-01, décret n°2-97-787 pour le Maroc ; directive 91/271/CEE et règlement (UE) 2020/741 pour l'Europe), et doivent être validées avant d'être présentées comme définitives.

5.2 Statistiques globales

Pour chaque paramètre temps réel, neuf indicateurs sont calculés sur la période sélectionnée :

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (\text{moyenne})$$

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (\text{écart-type d'échantillon})$$

Q_1 , médiane, Q_3 : 25^e, 50^e, 75^e percentiles

$$\text{IQR} = Q_3 - Q_1 \quad (\text{écart interquartile})$$

complétés par le minimum et le maximum observés.

5.3 Analyse scientifique (données temps réel)

Quatre sous-modules, accessibles par onglets.

5.3.1 Histogrammes

Répartition des valeurs d'un paramètre en 25 intervalles de largeur égale $\left(\frac{\max - \min}{25}\right)$, avec comptage du nombre de mesures par intervalle.

5.3.2 Corrélations (heatmap)

Matrice de corrélation entre les cinq paramètres, selon deux méthodes :

$$r_{\text{Pearson}} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$$

mesure une relation *linéaire* entre -1 et $+1$. La corrélation de **Spearman** applique la même formule aux *rangs* des valeurs plutôt qu'aux valeurs brutes, ce qui la rend sensible aux relations *monotones* (pas nécessairement linéaires) et plus robuste aux valeurs extrêmes.

5.3.3 Boxplots

Représentation de la médiane, du premier et troisième quartile (la « boîte »), des moustaches s'étendant jusqu'à $1,5 \times \text{IQR}$ au-delà de Q_1 et Q_3 , et des points au-delà de ces bornes signalés individuellement comme valeurs aberrantes.

5.3.4 Timeline (tendance)

Série brute superposée à une moyenne mobile sur une fenêtre glissante de N points (par défaut $N = 10$) :

$$\bar{x}_{\text{mobile}}(t) = \frac{1}{N} \sum_{k=0}^{N-1} x_{t-k}$$

permettant de visualiser une dérive lente au-delà du bruit mesure-à-mesure.

5.4 Comparaison

5.4.1 SET 1 vs SET 2

Test t de Welch. — Teste si les moyennes de deux échantillons diffèrent significativement, sans supposer l'égalité des variances :

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

Une valeur de $p < 0,05$ est interprétée comme une différence statistiquement significative entre les deux sets de capteurs.

Test de Kolmogorov-Smirnov (dérive). — Contrairement au test t qui ne compare que les moyennes, ce test compare la *distribution complète* des deux échantillons, par l'écart maximal entre leurs fonctions de répartition cumulées empiriques. Il permet de détecter une dérive progressive d'un capteur (encrassement, dérèglement) même lorsque la moyenne globale reste proche de l'autre capteur.

5.4.2 IN vs OUT_CONTROL vs OUT_PLANT

Comparaison des trois étapes du protocole expérimental (voir §5.8), utilisant les mêmes outils qu'à la section suivante (ANOVA, test de Tukey).

5.5 Alertes

5.5.1 Détection intelligente d'anomalies

Trois méthodes complémentaires sont proposées.

Z-score (univarié). —

$$z = \frac{x - \bar{x}}{s}$$

Une mesure est jugée anormale si $|z| > 3$ (seuil configurable). Cette méthode est simple et rapide, mais n'examine qu'un seul paramètre à la fois : elle peut ne pas détecter une combinaison de valeurs individuellement normales mais anormale collectivement.

Isolation Forest (multivarié). — Algorithme d'apprentissage non supervisé qui isole chaque observation en construisant des arbres de décision à séparations aléatoires. Un point anormal est statistiquement isolé en *moins d'étapes* qu'un point normal (il est plus facile à séparer du reste des données), car il se trouve dans une région moins dense de l'espace des cinq paramètres considérés conjointement. La contamination attendue est fixée à 5 % par défaut.

Autoencoder (multivarié). — Réseau de neurones entraîné à reconstruire son entrée après compression dans un espace de dimension réduite (goulot d'étranglement). Une observation anormale se reconstruit mal, produisant une erreur de reconstruction élevée. Le seuil de détection est fixé au 97,5^e percentile de l'erreur de reconstruction observée sur la fenêtre de données considérée.

5.5.2 Captures nulles consécutives

Ce mécanisme est **déjà entièrement implémenté côté équipe stockage** : dès qu'un capteur envoie N captures nulles consécutives (seuil configurable), une alerte de sévérité *warning* puis *critical* est levée automatiquement. Le module de monitoring se contente de consommer et d'afficher cette information, sans dupliquer la logique.

5.6 Journal

Calendrier mensuel dans lequel chaque jour ayant connu un événement est coloré selon la règle suivante :

- **Rouge** : au moins une alerte critique (captures nulles) *ou* une anomalie détectée par Isolation Forest ou Autoencoder (méthodes multivariées, jugées plus fiables) ;
- **Orange** : au moins une alerte de type *warning* *ou* une anomalie détectée uniquement par Z-score (méthode univariée, plus sensible mais moins spécifique).

Au clic sur un jour, le détail de chaque événement est affiché, incluant pour les anomalies la méthode responsable et une cause lisible (par exemple : « *SET 1 — pH : valeur 12,0 anormale-ment éloignée de la moyenne (z-score = 10,6, seuil = 3,0)* »). Pour des raisons de performance, la détection d'anomalies n'est exécutée que sur le mois actuellement affiché.

5.7 Qualité de l'eau

Synthèse, pour chaque paramètre, du taux de conformité aux normes sur l'ensemble de la période sélectionnée (répartition conforme aux deux normes / à une seule / à aucune), ainsi que du taux de complétude des données (pourcentage de mesures non manquantes).

5.8 Études et cycles

5.8.1 Rappel du protocole expérimental

IN (*Wastewater*) Eau usée brute, avant tout traitement.

OUT_CONTROL (*Control Series*) Sortie de la série de contrôle, **sans** plante.

OUT_PLANT (*Planted Series*) Sortie de la série **avec** plante.

Chaque mesure est répliquée trois fois par souci de fiabilité, sur huit paramètres (COD, BOD, TSS, pH, température, EC, turbidité, DO).

5.8.2 Onglet « Par cycle » — carte par paramètre

Pour chacun des huit paramètres, une carte combine :

- un boxplot à trois couleurs (une par étape du protocole) ;
- le résultat de l'**ANOVA à un facteur**, qui teste l'hypothèse d'égalité des moyennes des trois groupes :

$$F = \frac{\text{variance interclasse}}{\text{variance intraclasse}}$$

une valeur de $p < 0,05$ indique qu'au moins un groupe diffère significativement des autres ;

- la moyenne $\pm$ écart-type par étape ;
- le **pourcentage d'élimination**, calculé cycle par cycle puis moyenné :

$$\text{élimination}_c = \frac{\bar{x}_{IN,c} - \bar{x}_{\text{étape},c}}{\bar{x}_{IN,c}} \times 100$$

pour chaque cycle c , puis moyenne $\pm$ écart-type sur l'ensemble des cycles sélectionnés. Ce choix — plutôt qu'un calcul sur les moyennes globales poolées — traite chaque cycle comme une observation indépendante, ce qui est statistiquement plus rigoureux ;

- le **test post-hoc de Tukey HSD**, qui identifie précisément quelle paire de groupes diffère significativement (IN vs OUT_CONTROL, IN vs OUT_PLANT, OUT_CONTROL vs OUT_PLANT), avec p-value ajustée pour chaque comparaison.

5.8.3 Onglet « Progression multi-cycles »

Affiche, pour chaque paramètre, les boxplots des trois groupes côte à côte pour chaque cycle successif (cycle A, B, C, D, ...), permettant de visualiser si l'efficacité du traitement s'améliore, se stabilise, ou se dégrade au fil du temps.

5.8.4 Onglet « Statistiques globales »

Va au-delà des statistiques descriptives de base :

$$CV\% = \frac{s}{\bar{x}} \times 100 \quad (\text{coefficient de variation — dispersion relative})$$

$$IC_{95\%} = \bar{x} \pm t_{0,975, n-1} \times \frac{s}{\sqrt{n}} \quad (\text{intervalle de confiance de la moyenne})$$

où $t_{0,975, n-1}$ est le quantile à 97,5 % de la loi de Student à $n - 1$ degrés de liberté. S'y ajoutent l'**asymétrie** (skewness) et l'**aplatissement** (kurtosis), qui caractérisent la forme de la distribution, ainsi que le **test de normalité de Shapiro-Wilk**, qui indique si les données suivent une distribution normale — condition théorique de validité de l'ANOVA et du test t utilisés par ailleurs dans le tableau de bord.

5.8.5 Onglets « Histogramme » et « Heatmap de corrélation »

L'histogramme superpose la distribution des trois étapes pour un paramètre donné, afin de visualiser le déplacement de la distribution après traitement. La heatmap calcule la corrélation de Pearson entre les huit paramètres du cycle, en utilisant la moyenne des réplicats par couple (cycle, étape) comme unité d'observation, afin de repérer des relations entre paramètres (par exemple COD et BOD généralement très corrélés).

5.9 Tableau de données

Tableau paginé de l'intégralité des mesures brutes enregistrées depuis le début, filtrable par set de capteurs et par période, avec export CSV de tout l'historique ou d'une plage précise, pour un retraitement externe (Excel, SPSS, R).

CHAPITRE 6

Récapitulatif des méthodes statistiques

Le tableau 6.1 synthétise l'ensemble des méthodes de calcul utilisées dans le module de monitoring.

Table 6.1: Récapitulatif des formules et méthodes statistiques utilisées

Méthode	Principe / formule	Utilisation
Moyenne	$\bar{x} = \frac{1}{n} \sum x_i$	Partout
Écart-type	$s = \sqrt{\frac{1}{n-1} \sum (x_i - \bar{x})^2}$	Partout
Médiane, Q_1 , Q_3	Percentiles 50/25/75	Boxplots, statistiques
IQR	$Q_3 - Q_1$	Détection d'outliers
Règle des outliers	Hors $[Q_1 - 1,5 \text{ IQR}, Q_3 + 1,5 \text{ IQR}]$	Boxplots
Z-score	$z = (x - \bar{x})/s$	Anomalies univariées
Coeff. de variation	$CV\% = (s/\bar{x}) \times 100$	Dispersion relative
Intervalle de confiance 95%	$\bar{x} \pm t_{0,975,n-1} s/\sqrt{n}$	Fiabilité de la moyenne
Corrélation de Pearson	$\text{cov}(x, y)/(s_x s_y)$	Relations linéaires
Corrélation de Spearman	Pearson sur les rangs	Relations monotones
Test t de Welch	$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$	Comparaison de 2 groupes
ANOVA (F)	Variance interclasse / intraclasse	Comparaison de 3+ groupes
Tukey HSD	Post-hoc, comparaisons par paires	Localiser la différence
Kolmogorov-Smirnov	Écart max. entre fonctions de répartition	Dérive de distribution
Shapiro-Wilk	Test de normalité	Validité des tests paramétriques
Skewness / Kurtosis	Moments d'ordre 3 et 4	Forme de la distribution
Isolation Forest	Isolement par arbres aléatoires	Anomalies multivariées
Autoencoder	Erreur de reconstruction	Anomalies multivariées
ACP (PCA)	Projection sur axes de variance maximale	Réduction de dimension
% d'élimination	$(\bar{x}_{IN} - \bar{x}_{étape})/\bar{x}_{IN} \times 100$	Efficacité du traitement

6.0.1 Analyse en Composantes Principales (ACP)

L'ACP réduit un ensemble de paramètres corrélés à un petit nombre d'axes (« composantes ») qui concentrent l'essentiel de la variance des données, ce qui permet de visualiser en deux

dimensions des données qui en comptent initialement cinq à huit.

Principe. — Les paramètres, exprimés dans des unités hétérogènes (mg/L, °C, µS/cm, NTU...), sont d’abord standardisés (moyenne nulle, écart-type unitaire) pour qu’aucun ne domine artificiellement l’analyse du fait de son échelle :

$$X_{std} = \frac{X - \mu}{\sigma}$$

La matrice de covariance de X_{std} est ensuite diagonalisée pour obtenir ses axes de variance maximale (les composantes principales), sur lesquels les observations sont projetées. La part de variance expliquée par la i -ème composante est :

$$\tau_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

où λ_i est la i -ème valeur propre de la matrice de covariance.

Implémentation. — Calculée côté `monitoring_backend` via `StandardScaler` et PCA de `scikit-learn`, sur deux jeux de données distincts :

- GET `/studies/{study_id}/comparison/pca` — cycles de laboratoire, une observation par couple (cycle, étape), sur les huit paramètres du protocole ;
- GET `/api/monitoring/analytics/pca` — capteurs temps réel, une observation par relevé horodaté, sur les cinq paramètres numériques.

Dans les deux cas, l’ACP n’est calculée que si au moins 3 observations et 2 paramètres restent disponibles après suppression des valeurs manquantes ; en-deçà, un message d’erreur explicite est retourné plutôt qu’un résultat non significatif.

Restitution. — Le tableau de bord affiche un nuage de points des observations projetées sur les deux premières composantes (PC_1 , PC_2), coloré par étape du protocole ou par set de capteurs selon le contexte, accompagné des pourcentages de variance expliquée et d’un tableau des *loadings* — la contribution de chaque paramètre à chaque composante, utile pour interpréter la signification physique des axes (un axe dominé par la DCO et la turbidité s’interprète par exemple comme un axe de charge organique).

CHAPITRE 7

Limites connues et perspectives d'amélioration

- **Absence de persistance des modèles de machine learning** : l'Isolation Forest et l'autoencodeur sont ré-entraînés à chaque appel, sur la fenêtre de données demandée, sans mécanisme de *checkpoint*. Ceci reste largement suffisant pour le volume de données actuel, mais devrait être reconsidéré si le trafic augmentait significativement.
- **Filtrage temporel des données brutes** : reconstruit côté monitoring par pagination successive, l'API de l'équipe stockage ne proposant pas nativement de filtre par date. La solution fonctionne correctement mais devient coûteuse sur de très grands volumes.
- **Module de prévision (LSTM)** : prévu au cahier des charges mais non traité dans ce lot de travail, celui-ci étant resté centré sur les neuf modules du tableau de bord demandés initialement.
- **Seuils de conformité réglementaire** : les valeurs utilisées pour les badges de conformité sont indicatives et doivent être validées par le chef de projet avant toute utilisation officielle.

CHAPITRE 8

Conclusion

Le module de monitoring développé répond à l'ensemble des exigences fonctionnelles définies dans le cahier des charges : visualisation temps réel, statistiques descriptives et avancées, analyse scientifique multi-méthodes, détection intelligente d'anomalies combinant approches statistique et machine learning, comparaison rigoureuse des performances de traitement par des tests d'hypothèses appropriés, et un journal chronologique centralisant l'ensemble des incidents détectés.

L'architecture retenue — séparation stricte entre stockage, calcul et présentation, communication exclusivement par API REST — garantit l'indépendance du module vis-à-vis des autres composants du système, tout en assurant la traçabilité et la reproductibilité de chaque indicateur affiché, chacun étant adossé à une méthode statistique documentée et une justification technique explicite.